

The Basics of Object-Oriented Programming

SWEN1 | BIF3

FH University of Applied Sciences
TECHNIKUM WIEN

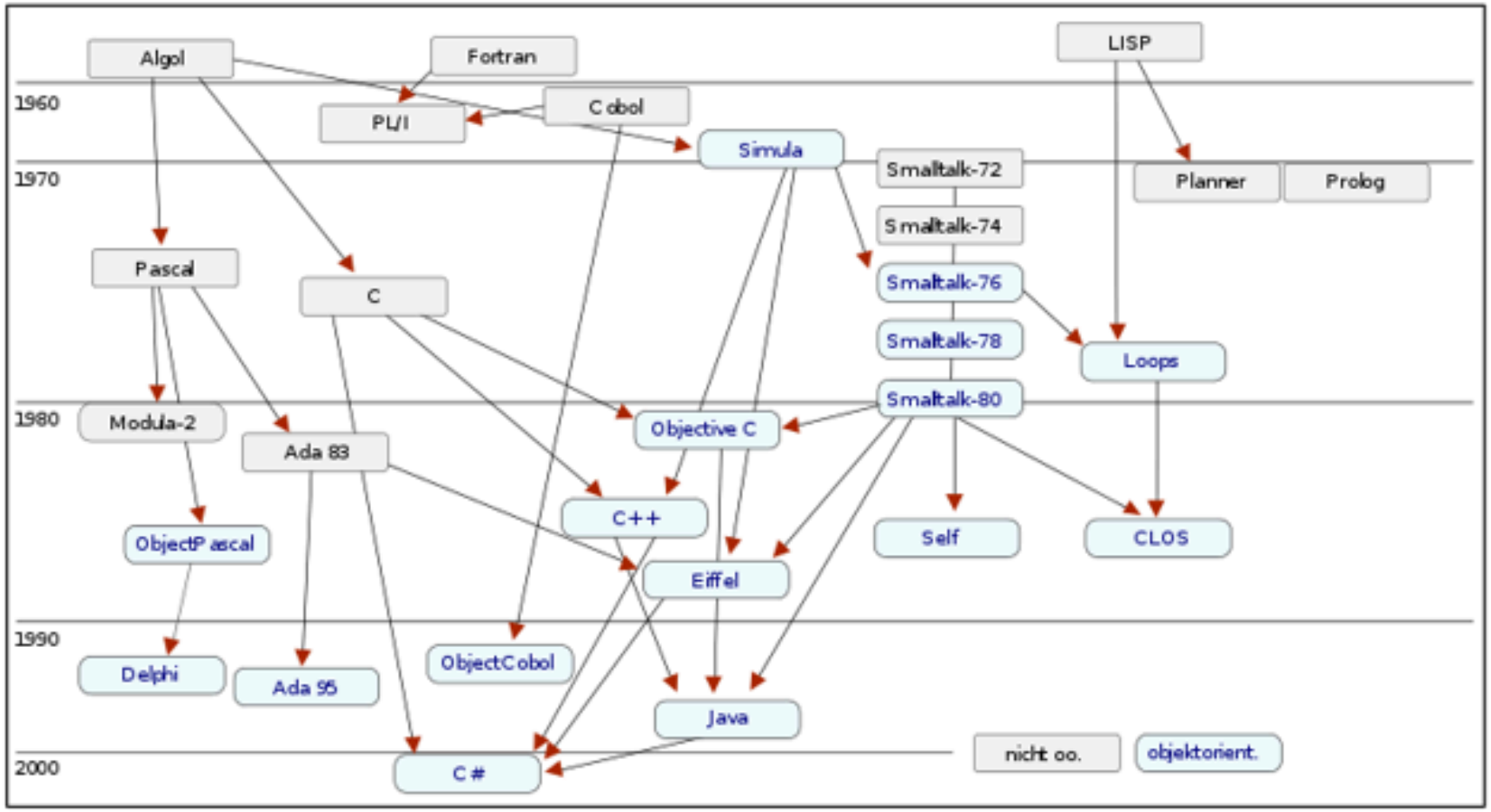
Building blocks of Object-Oriented Modeling

We create **objects** based on **classes** by using

- **Abstraction**:
  - An object from real world is represented in an object of the model
  - Not all aspects are taken over
  - Essential aspects are defined (common characteristics)
- **Encapsulation**
  - Hides internals of an object from public
  - Makes it possible to change internals without affecting others
- **Modularity**
  - Divide and conquer – approach
  - Splits complex objects in understandable small pieces
- **Inheritance**: You can base classes on other classes, leads to a hierarchy of classes
- **Polymorphism**: The ability to hide many different implementations behind a single interface

OOP Starting

- in 1960s with SIMULA (a simulation language)
- most of the concepts alread in place
- Lot of programming-languages followed:
  - Smalltalk (1970s)
  - Ada (1980s)
  - C++ (1980s)
  - Eiffel (1980s)
  - Java (1990s)
  - C# (2000s)



OOP Terms

- **Class**: blue-print for objects, defines data and behavior
- **Object**: instance, size in memory with data
- **Member**: all items of a class
  - **Method**: a function as part of a class
  - **Field**: a variable as part of a class
  - **Property** – Attention: can be a misleading term
    C# = language construct to get/set methods to expose fields
    Java = a util class to read settings from a .properties-file (get-set method-pair in java done using functions, pattern-support in IDE exists)
- **Event**

Class

A class is a blue-print to instantiate objects. Hereby it defines the common attributes/properties (which one, not the value itself; e.g. speed; not 20 km/h) and the operations/behavior that can be performed for all instances of a class. Furthermore it defines relations (links = instance of an association) to other classes and semantics.

e.g.: a human should have the following **properties or characteristics**: a name and a speed (Nouns!)
Human(Name, Speed)

In contrast an instance of a class human e.g.: “Jack Bauer” has concrete values to these members e.g.: (“Jack Bauer”,“20km/h”).

We see here that a class is an abstraction which is modeled for a purpose, so that it highlights important aspects and ignore irrelevant ones.

Classes in Class-Diagrams

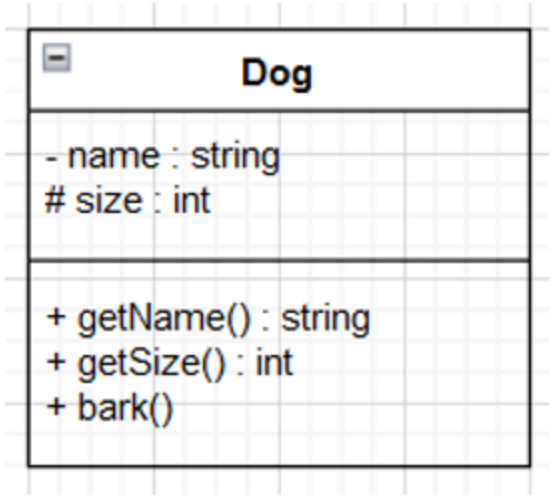
Split up in 3 sections:

- Class **Name**
- **Structure** (n attributes --> variable of class with datatype/access modifier)
- **Behavior** (n operations --> execute behavior based on own attributes)

UML Notation



Alternative:



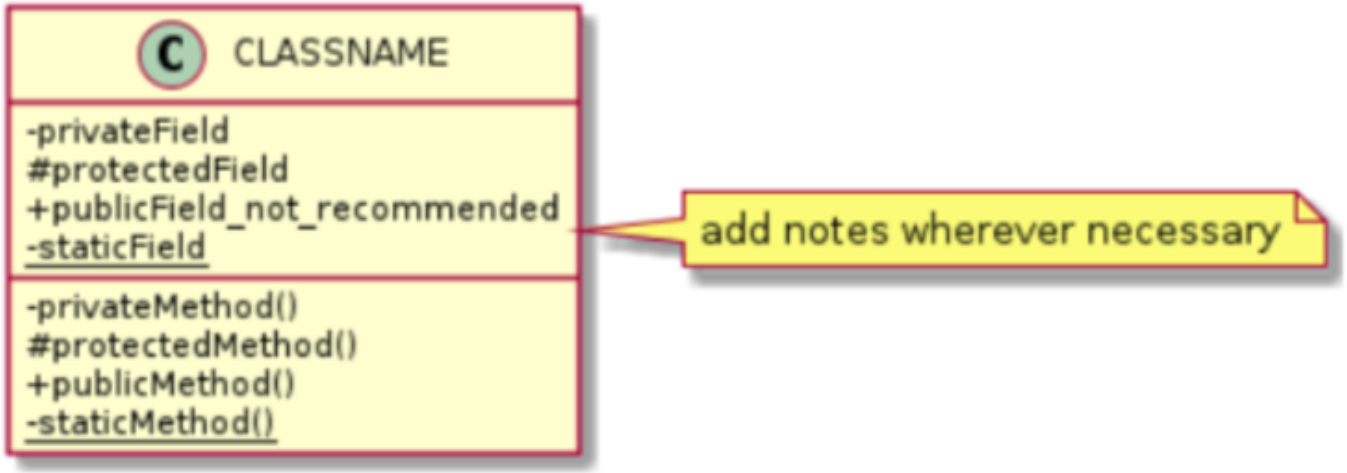
PlantUML Code:

```
@startuml
class Dog {
   - name
   # size
   + getName()
   + getSize()
   + bark()
}
@enduml
```

Building Blocks

- **Member variable** (member fields, -properties, -attributes)
  - Class variables: static member variables (1 copy at all)
  - Instance variables: each object has independent store (1 copy per object)
- **Methods**
  - Class methods
  - Instance Methods

Note: in case the state or behavior is not related to an object, but to a class, we can define **„static“** member. (1 static state is shared across the app.)



Samples of a class

```csharp
// C#
public class ExampleClass
{
    // static member
    private static int _creationCount = 0;

    // instance member
    private int _delta = 0;

    // instance auto-property
    public bool IsFine { get; set; } = false;

    // constructor
    public ExampleClass(int delta) {
        _creationCount++;
        this._delta = delta;
    }

    // instance method
    public int GetDelta()
    {
        return this._delta;
    }
}
```

```java
// Java
public class ExampleClass {
    // static member
    private static int creationCount = 0;

    // instance member
    private int delta = 0;

    // (backing) field
    private boolean fine = false;
    public boolean IsFine() {
        return fine;
    }
    public void setFine(boolean fine) {
        this.fine = fine;
    }

    // constructor
    public ExampleClass(int delta) {
        creationCount++;
        this.delta = delta;
    }

    // instance method
    public int GetDelta()
    {
        return this.delta;
    }
}
```

Object

... is an instance of a class with a given *identity* = unique identification, so that 2 objects with same state have still different identities.

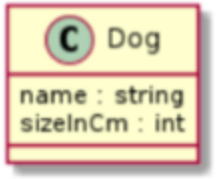
- Attributes (state)
- Methods (behavior; can change state)

Hint: in debugger we distinguish identities by memory address.

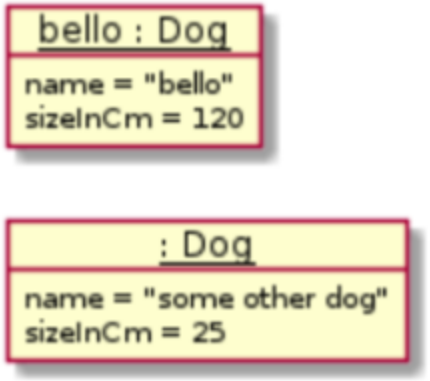
Objects in Object-Diagrams

- object is depicted as a rectangle with its name underlined
- are used in object diagrams (current state of a system (=snapshot)) Attention: often mixed up with class diagrams.
- can have a name or be an anonymous object

class diagram



object diagram



Immutable objects: if state cannot be changed after creating the object

Goals of OOP

- design classes not too big (god-class) and not too small (not maintainable)
- may refer to objects of the real world
- keep the object in a valid state
- keep corresponding methods near to the data (so method can use its object-data)
  - design your classes highly cohesive
- loose coupled
- testable



Object-Oriented Programming by Example

SWEN1 | BIF3

FH University of Applied Sciences
TECHNIKUM WIEN

Example: Chess figures

Some instances of one class:



Question: Data & Behavior

- How would you call the class?
- Which attributes would you define?
- Which methods would you define to be able to use the instances inside their system?

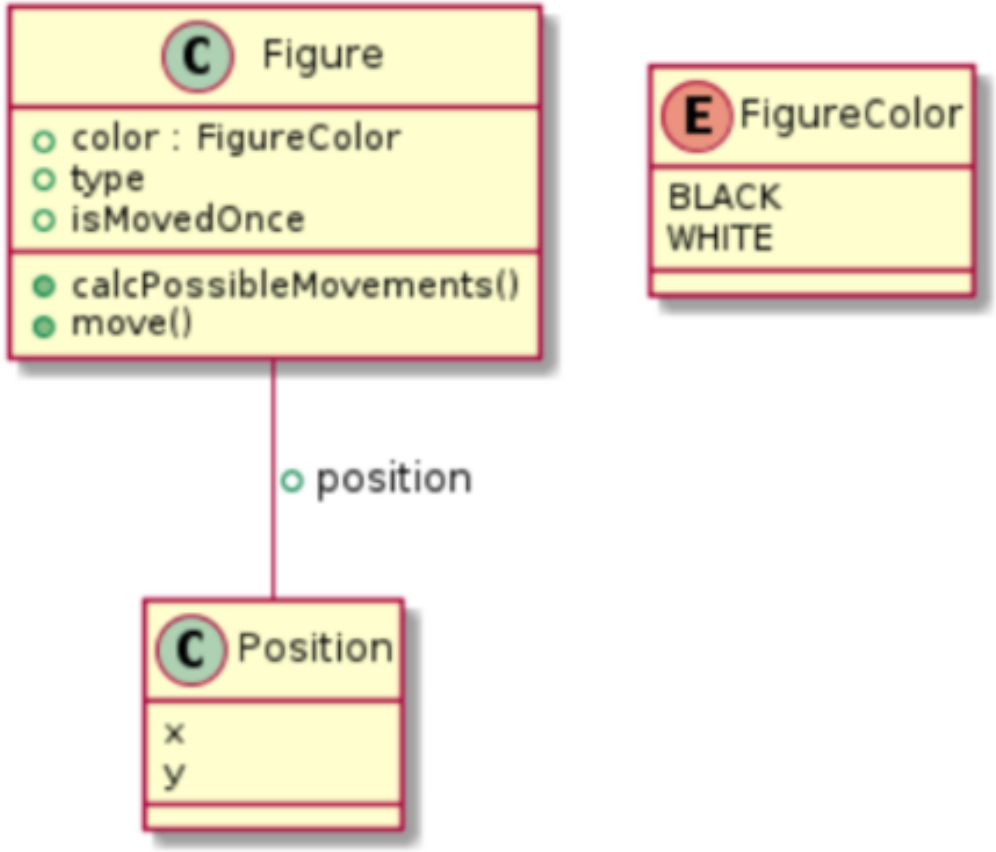
Solution: The „Figure“

Figure data:

- Color (discrete value: black or white)
- Type (discrete value: king, rook, bishop, queen, knight, pawn)
- Position (x and y value 1-8, A-H)
- Is Moved Once (castling (de: Rochade))
- …

Figure methods:

- Calculate Possible Movements
- Move
- …



C# Chess Figure Sample

```
public class ChessFigure
{
    public string Type { get; } // consider enum
    public int[] Position { get; } = new int[2];    // else X and Y prop
    public bool IsWhite { get; }
    public bool IsAlive { get; set; }
    private ChessBoard _board;

    public ChessFigure(ChessBoard board, string type, bool isWhite, int[] startPos)
    {
        // ...
    }

    public void MoveTo(int[] targetPosition)
    {
        // fancy move logic with hitting figures from "Board" and set IsAlive
    }
}
```

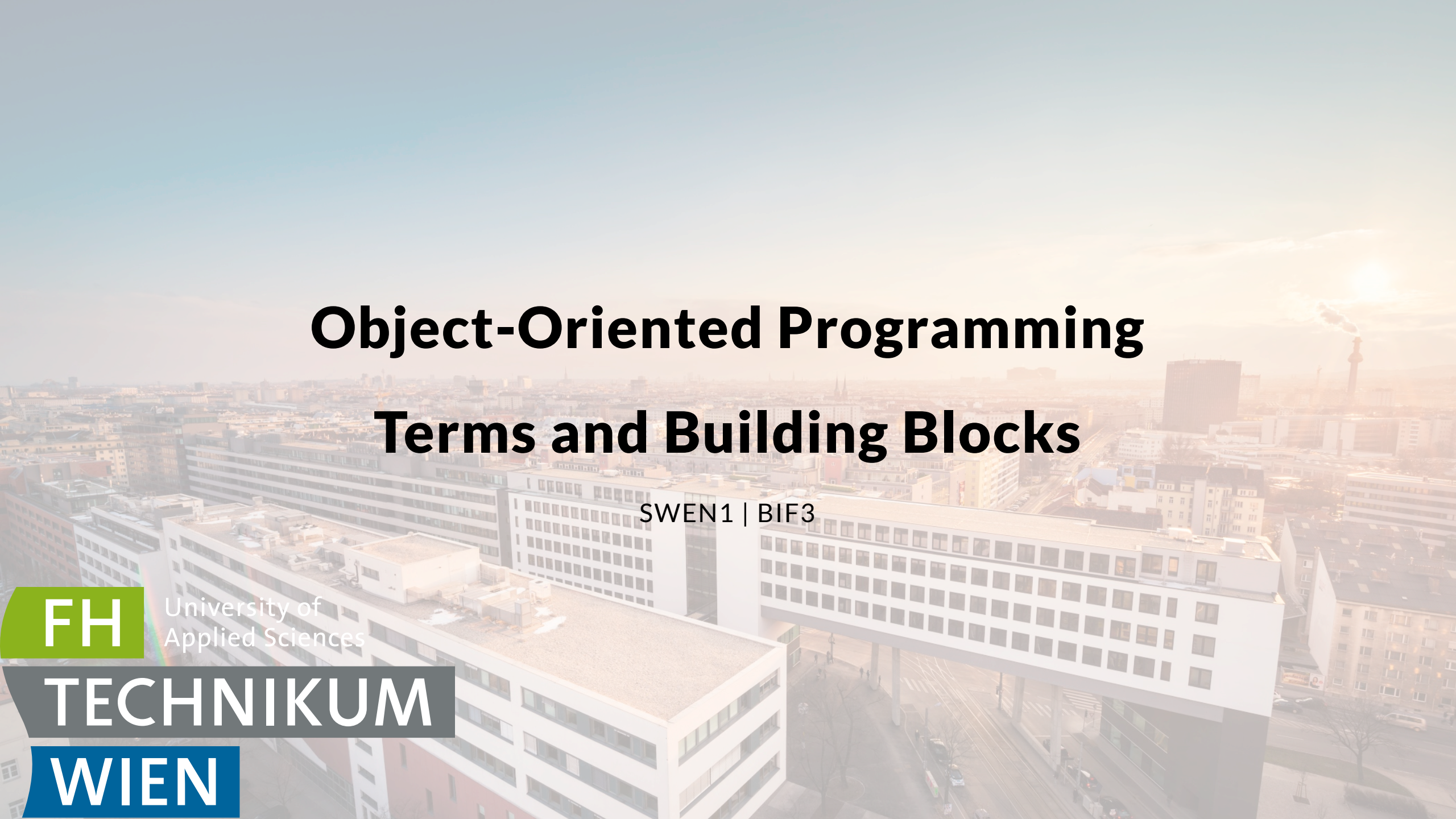
Java Chess Figure Sample

```java
public class ChessFigure {
    private String type; // consider enum
    private int[] position = new int[2];    // else X and Y prop
    private boolean isWhite;
    private boolean isAlive;
    private ChessBoard _board;

    public ChessFigure(ChessBoard board, String type, boolean isWhite, int[] startPos) {
        // ...
    }

    public void MoveTo(int[] targetPosition)
    {
        // fancy move logic with hitting figures from "Board" and set IsAlive
    }

    public String getType() { return type; }
    public int getPosition() { return position; }
    public boolean isWhite() { return isWhite; }
    public boolean isAlive() { return isAlive; }
}
```



Object-Oriented Programming

Terms and Building Blocks

SWEN1 | BIF3

FH University of Applied Sciences
TECHNIKUM WIEN

Encapsulation

Problem:

- if the state is directly accessible from outside your state can (potentially) be corrupted

Solution:

- public interface of a class published to their users should be kept constant (=contracts)
  - defines how to use the class
  - defines how to test the class
  - extensions are (often) no problem
- inner structure/implementation is hidden and not accessible from outside (information hiding; Parna's principle)
  - (refactorings and optimizations can be done on data and methods)

Encapsulation

- class is responsible to protect data integrity/consistency
  - class can trust the internally managed state
- best practice is to read/write data via methods
  (so that data can not be set directly; checks can be performed before data is actually taken over)
- access control enforced by compiler (see access-modifier)
  - else everybody can use a class (no information / restriction)

Hint: some programming languages have already defined constructs for such access: e.g.: C#
“Properties” (get/set) -> equivalent in java to getter and setter methods)

Access

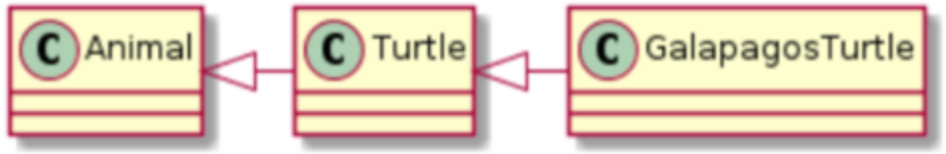
- **public** (+): everybody can access the item directly
- **protected** (#): only from inside the class and from inherited classes (see next chapter) the items can be accessed directly, but hidden for usage from outside
- **private** (-): only from inside the class the items can be accessed directly
- *Default*: when a access-modifier is omitted:
  - Java: defaults to **package-private** meaning that it is only accessible from within the same package
  - C#: defaults to **internal** meaning it is only accessible within the same assembly

Inheritance

- (directional) **“is-a”** relationship between classes (extensibility)
- concepts: single-inheritance (C#, Java) or multi-inheritance (C++)
- specified members (attributes, operations, relations) are taken over from parent class to child class (reusability and DRY)
- members can be
  - **extended** (further items added)
  - **overridden** (adapted in child-class, but keep code even with child-class working; see further reading: Liskov’s substitution principle)
  - **overlapped** (also shadowing; add items named exactly the same with same parameters as in the base-class; as in base-class; compiler can not distinguish here)
  - **overloaded** (add items named exactly the same with different parameters as in the base-class; compiler can distinguish which method is meant)

Inheritance in Class-Diagrams

- super class, base class, parent class: Animal
- sub class, derived class, child class: Turtle



We are not limited to one level of inheritance, we often use the terms

- ancestors (de: “Vorfahren”) and
- descendant (de: “Nachfahre”)

to talk about all classes that are in the inheritance hierarchy lower or higher to a specific class.

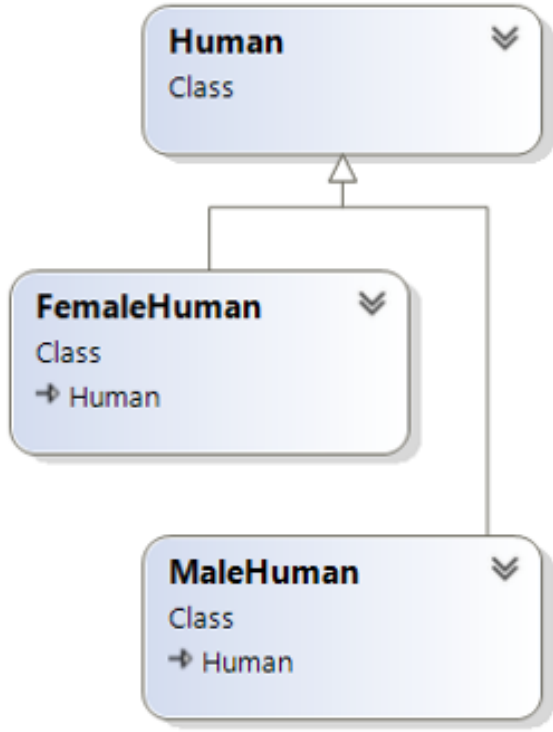
Attention: UML does not limit you to single inheritance (but C# / Java do)

Polymorphism

Here we can access the derived object by using a reference to an (explicitly implemented) base-class type (single interface) (see: subtyping).
This means that dependent of the subtype the original or the overridden method is chosen.

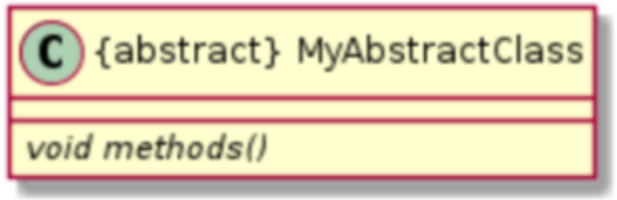
```
FemaleHuman she = new FemaleHuman();
Human someone = she;  // implicit cast
FemaleHuman marry = (FemaleHuman)someone; // explicit cast
```

Because a FemaleHuman is a human we can assume that all operations of a Human are also implemented in FemaleHuman. We can reference an object of type FemaleHuman also as Human.



Abstract

When a class is defined **abstract** it is not allowed to create an instance of this class. It can only be used as a base-class for further inheritance steps. Hereby the defined abstract methods need to be implemented (except the child-class itself is again abstract).
A class needs to be abstract if at least one member is defined abstract.

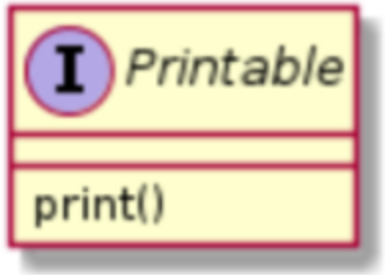


Forbid further inheritance: define a class as not “inheritable”

- Java: **final** class
- C#: **sealed** class

Interface

- Define a contract to implement, but without any implementation. A class can implement many different interfaces.
- You can not instantiate an interface.
- Similar concept but completely the opposite is an abstract class.
- (often in UML drawn with «interface» -information above the name)



Abstract Class vs Interface

Abstract class:

- When we need to implement a base type that knows how to implement some part of the abstraction i.e. partial implementation and the other/remaining part of the abstraction cannot be implemented by this class.
- --> WHAT KIND OF OBJECT IS THIS?

Interface:

- When we only need to enforce a contract. Whoever will implement this interface will provide an implementation of the methods.
- --> WHAT SHOULD THIS OBJECT BE CAPABLE OF?

Important rule for Developers

==> Program to interfaces, not implementations!

- loose coupling
- component-based programming
- easier maintainability
- makes your code base more scalable
- makes code reuse much more accessible, because the implementation is separated from the interface